
LÄR DIG AI FRÅN GRUNDEN - TILLÄMPAD MASKININLÄRNING MED PYTHON

Antonio Prgomet
Terese Johnson
Amanda Solberg
Linus Rundberg Streuli

Detta verk är skyddat av upphovsrättslagen.

Den som bryter mot upphovsrättslagen kan åtalas av allmän åklagare och dömas till böter eller fängelse i upp till två år samt bli skyldig att erlägga ersättning till upphovsman eller rättsinnehavare.

© Pedagogicus Publishing

Innehållsförteckning

Förord	7
Bokens hemsida	7
Bokens målgrupp	7
Element i boken	7
Språk	8
Bokens upplägg	8
Lycka till	9
 I Introduktion till maskininläring	 11
1 Introduktion till maskininläring	13
1.1 Artificiell intelligens (AI), maskininläring (ML) och djupinläring (DL)	14
1.2 Fyra problemkategorier inom maskininläring: regression, klassificering, dimensionsreducering och klustring	16
1.3 Fundamentala koncept	19
1.3.1 Träningsdata, valideringsdata och testdata	21
1.3.2 K-delad korsvalidering	30
1.3.3 <i>RMSE</i>	33
1.3.4 Hyperparametrar och parametrar	35
1.3.5 <i>Grid search</i>	36
1.3.6 Kategorisk data	42
1.3.7 <i>Feature engineering</i>	47
1.4 Tvärsnittsdata, tidsseriedata och paneldata	51
1.5 Övningsuppgifter	54

2	Ett ML-projekt från början till slut	55
2.1	En checklista för Maskininlärning	55
2.1.1	Definiera problemet och skapa en helhetsbild	56
2.1.2	Få tillgång till datan	56
2.1.3	Utforska datan, gör en <i>exploratory data analysis (EDA)</i>	57
2.1.4	Bearbeta datan	58
2.1.5	ML-modellering	59
2.1.6	Presentera din lösning för intressenter	60
2.1.7	Produktionssättning av modellen och övervakning av implementeringen	60
2.2	Ett kodexempel från början till slut - Huspriser i Kalifornien	63
2.3	Utmaningar inom ML	90
2.4	<i>Scikit-learn</i>	90
2.4.1	Designprinciper	92
2.4.2	<i>Estimators, predictors, transformers</i>	95
2.4.3	<i>Pipelines</i>	97
2.4.4	Ett avstick till <i>TensorFlow</i> och <i>Keras</i>	102
2.5	Övningsuppgifter	102

II Maskininlärning 103

3	Regression	105
3.1	Regressionsproblem	105
3.2	Utvärderingsmått	106
3.2.1	<i>Root mean squared error (RMSE)</i>	106
3.2.2	<i>Mean squared error (MSE)</i>	107
3.2.3	<i>Mean absolute error (MAE)</i>	107
3.3	Regressionsmodeller	109
3.3.1	Linjär regression	111
3.3.2	Optimeringsalgoritm - <i>Gradient descent</i>	122
3.3.3	<i>The bias variance trade-off</i>	128
3.3.4	<i>Ridge</i> regression (L2-regularisering)	129
3.3.5	<i>Lasso</i> regression (L1-regularisering)	133
3.3.6	<i>Elastic net</i> (kombination av <i>Lasso</i> & <i>Ridge</i>)	135
3.3.7	<i>Support vector machines (SVM)</i>	136
3.3.8	Beslutsträd	139
3.3.9	<i>Ensemble learning</i>	144
3.3.10	<i>Random forest</i>	149

3.4	Övningsuppgifter	150
4	Klassificering	151
4.1	Klassificeringsproblem	151
4.1.1	<i>OvR</i> - och <i>OvO</i> -algoritmerna	154
4.2	Utvärderingsmått	154
4.2.1	<i>Confusion matrix</i>	155
4.2.2	<i>Accuracy</i>	158
4.2.3	<i>Precision</i> och <i>Recall</i>	159
4.2.4	<i>F1-score</i>	162
4.2.5	<i>ROC</i> -kurvan	164
4.3	Klassificeringsmodeller	166
4.3.1	Logistisk regression	168
4.3.2	<i>Support vector machines (SVM)</i>	176
4.3.3	Beslutsträd	190
4.3.4	<i>Ensemble learning</i>	195
4.3.5	<i>Random forest</i>	203
4.4	Två kodexempel	206
4.4.1	Kodexempel 1 - Klassificering av <i>MNIST</i>	206
4.4.2	Kodexempel 2 - Välja önskad <i>precision</i> och <i>recall</i>	213
4.5	Övningsuppgifter	216
5	Dimensionsreducering	217
5.1	<i>Curse of dimensionality</i>	217
5.2	Vad är dimensionsreducering?	220
5.2.1	Effekter av dimensionsreducering	220
5.3	Principalkomponentanalys (<i>PCA</i>)	223
5.3.1	<i>Kernel PCA</i>	229
5.4	Övningsuppgifter	232
6	Klustering	233
6.1	Vad är klustering?	233
6.1.1	Exempel på tillämpningsområden för klustering	234
6.2	<i>K-Means</i>	237
6.2.1	Hur genomförs klustering med <i>K-Means</i> ?	240
6.2.2	Hur tränas modellen?	243
6.2.3	Val av antalet kluster	246
6.3	Två kodexempel	252
6.3.1	Kodexempel 1 - Kundsegmentering	252

6.3.2	Kodexempel 2 - Semi-vägledad inlärning	256
6.4	Övningsuppgifter	259

III Djupinlärning 261

7	Artificiella neurala nätverk (ANN)	263
7.1	Bakgrund	264
7.2	Modellarkitektur	264
7.2.1	Hur fungerar ett neuralt nätverk	265
7.2.2	Riktlinjer för modellarkitektur	272
7.2.3	Intuition för antal layers och noder	274
7.3	Regularisering	275
7.4	Träning av neurala nätverk - <i>Backpropagation</i>	280
7.5	Två kodexempel	284
7.5.1	Kodexempel 1 - Bildklassificering av <i>Fashion MNIST</i>	284
7.5.2	Kodexempel 2 - Optimering av hyperparametrar med <i>KerasTuner</i>	292
7.6	Övningsuppgifter	296
8	<i>Convolutional neural network (CNN)</i>	297
8.1	Bakgrund	297
8.2	Hur fungerar <i>CNN</i> ?	298
8.2.1	<i>Convolution layers</i>	298
8.2.2	<i>Pooling layers</i>	301
8.2.3	<i>Padding</i>	301
8.3	Exempel på en modellarkitektur för <i>CNN</i>	302
8.4	<i>Data augmentation</i>	305
8.5	Tre kodexempel	305
8.5.1	Kodexempel 1 - Bildklassificering av <i>CIFAR-100</i>	306
8.5.2	Kodexempel 2 - Förtränade modeller	310
8.5.3	Kodexempel 3 - <i>Transfer learning</i>	312
8.6	Övningsuppgifter	316
9	<i>Recurrent neural network (RNN)</i>	317
9.1	Bakgrund	317
9.2	Modellarkitekturer	319
9.2.1	<i>Long short-term memory (LSTM)</i>	320
9.2.2	<i>Gated recurrent unit (GRU)</i>	320
9.3	<i>Natural language processing (NLP)</i>	321

9.3.1	<i>Embeddings</i>	321
9.4	Kodexempel - Sentimentanalys	323
9.5	Övningsuppgifter	327
10	Chattbottar	329
10.1	ChatGPT och <i>prompt engineering</i>	329
10.2	Molnbaserade och lokala språkmodeller	330
10.2.1	Molnbaserade modeller	331
10.2.2	Lokala modeller	331
10.3	Bygga en chattbot	331
10.4	<i>Retrieval Augmented Generation (RAG)</i>	333
10.4.1	Läs in data	334
10.4.2	Chunking	335
10.4.3	<i>Embeddings</i>	336
10.4.4	Semantisk sökning	337
10.4.5	Generera svar	340
10.4.6	Evaluering	341
10.5	Vector store	343
10.6	Vidare arbete med chattbottar	346
10.7	Övningsuppgifter	346

Förord

Bokens hemsida

Boken har en tillhörande hemsida där material kopplat till boken finns uppladdat.
https://github.com/AntonioPrgomet/ai_tillaempad_ml

Bokens målgrupp

Boken är avsedd för kurser av olika slag och används bland annat inom yrkeshögskolan, andra läroinstitut, företag och organisationer. Boken går även utmärkt att använda för självstudier.

Boken förutsätter att läsaren har grundläggande kunskaper i pythonprogrammering. Om det saknas går boken fortfarande att läsa efter att man gjort grundkursen i Python som finns tillgänglig på bokens hemsida. På bokens hemsida finns det även ett dokument där summasymbolen (Σ) samt grundläggande vektor- och matrisalgebra går igenom. Den läsare som saknar de kunskaperna kan läsa igenom det dokumentet.

Boken kan användas på olika sätt.

- En elementär kurs i AI/ML kan inkludera kapitel 1-4.
- En grundkurs i AI/ML kan inkludera kapitel 1-6.
- En grundkurs i AI/Djupinlärning kan inkludera kapitel 1-2 och kapitel 7-10.
- En heltäckande kurs inom ML kan inkludera hela boken, kapitel 1-10.

Element i boken

Boken använder sig av två element som är värda att notera.

Det finns informationsrutor som ser ut enligt nedan.

i Så här ser en informationsruta ut. I de här rutorna skriver vi korta fördjupande förklaringar, reflektioner och kommentarer.

Ofta är bokens kodexempel annoterade. På den högra kanten förekommer siffror som motsvarar en förklarande text under kodexemplet. Efter den förklarande texten följer utskrifter från själva koden. Vi ser ett exempel här nedanför.

```
print('An annotated code example.')
```

①

① Siffran (1) till höger i kodexemplet motsvaras av en förklarande text direkt under kodexemplet.

An annotated code example.

Språk

Boken är skriven på svenska. Bokens kodexempel är däremot på engelska eftersom det är en stark praxis världen över att kod skrivs på engelska. Vi har använt svensk terminologi i största möjliga utsträckning men vissa ord, exempelvis *pipeline* brukar benämnas så även i svenska och därför har vi behållit det. Engelska ord *kursiveras* i texten.

Bokens upplägg

Boken är uppdelad i tre delar.

1. Introduktion till maskininlärning. Här introduceras maskininlärning där vi får en helhetsbild av ämnet och lär oss fundamentala koncept som används genom hela boken.
2. Maskininlärning. Här lär vi oss grunderna inom maskininlärning som består av de fyra områdena regression, klassificering, dimensionsreducering och klustring. Vi använder primärt biblioteket *scikit-learn* för modellering.
3. Djupinlärning. Här lär vi oss grunderna inom djupinlärning där modeller som kallas för neurala nätverk används. Djupinlärning är ett delämne inom maskininlärning. Här använder vi primärt biblioteken *TensorFlow* och *Keras* för modellering.

Lycka till

AI och maskininlärning är högaktuella ämnen och kunskaper inom dem kan skapa många möjligheter. Vi författare hoppas att du ska tycka att boken är lika rolig att läsa som vi har tyckt det är att skriva den.

Lycka till!

Antonio Prgomet - <https://www.linkedin.com/in/antonioprgomet/>

Terese Johnson

Amanda Solberg

Linus Rundberg Streuli

Del I

Introduktion till maskininlärning

Kapitel 1

Introduktion till maskininlärning

I detta kapitel kommer vi att få en första helhetsbild över maskininlärning. Vi kommer börja med att lära oss hur artificiell intelligens (AI), maskininlärning (ML) och djupinlärning (DL) hänger ihop. Därefter fortsätter vi att kolla på fyra centrala problemkategorier inom ML, fundamentala koncept som vi kommer ha nytta av genom hela boken och avslutningsvis kollar vi på hur data kan kategoriseras som tvärsnittsdata, tidsseriedata och paneldata.

I nästa kapitel kommer vi gå igenom ett ML-projekt från början till slut vilket ytterligare konkretiserar den helhetsbild vi får i detta kapitel. Dessa två kapitel har alltså en nära koppling till varandra. Under den första läsningen av dessa två kapitel bör läsaren inte försöka förstå alla detaljer eftersom många av koncepten kommer användas senare in i boken och då bli mer naturliga. Det är därför en god idé att göra en första genomläsning och försöka få en helhetsbild, när man sedan kommit längre in i boken kan man återkomma för att repetera och testa sin förståelse.

1.1 Artificiell intelligens (AI), maskininlärning (ML) och djupinlärning (DL)

AI är ett relativt nytt ämne och begreppet *artificiell intelligens* myntades först 1956. Men vad handlar AI om? AI handlar om förmågan hos datorprogram och robotar att efterlikna människors eller djurs intelligens. Några exempel på tillämpningsområden, som vi kommer lära oss i denna bok är:

- Utföra prediktioner. Exempelvis prediktera en persons inkomst baserat på dennes ålder och utbildningsnivå.
- Datorseende. Exempelvis kunna se vad det är för siffra någon skrivit på ett papper.
- Kommuniera. Exempel på detta är chatbottar såsom ChatGPT.

AI som ämne har flera delämnen där ML utgör en central del. Det finns flera definitioner och nedan ser vi två vanligt förekommande:

“Machine Learning is the field of study that gives computers the ability to learn without being explicitly programmed” (Arthur Samuel, 1959).

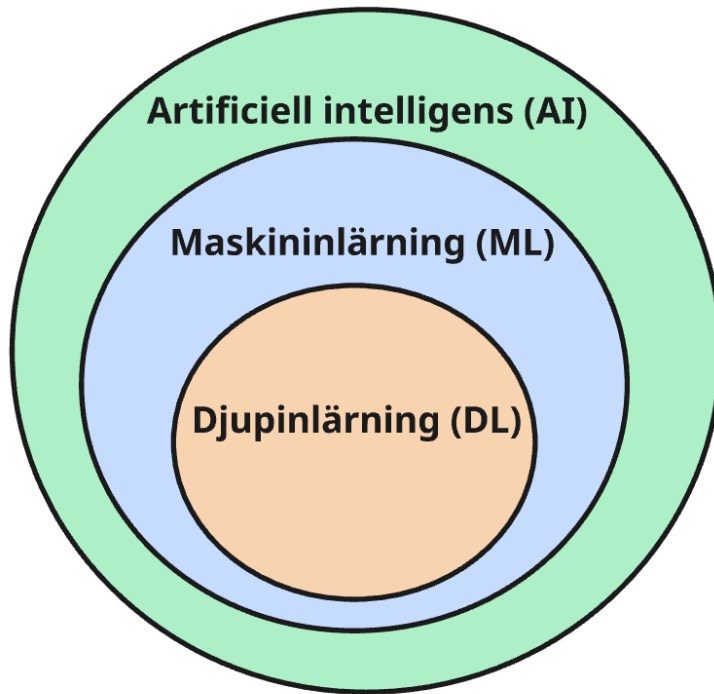
“The science and art of programming computers so they can learn from data” (Tom Mitchell, 1997).

Den första definitionen av Samuel betonar datorers förmåga *att lära sig* utan att vara explicit programmerade. Ett exempel är att vi baserat på data om människors löner och åldrar kan få datorn att lära sig att förstå vilket samband som råder mellan lön och ålder. Därefter kan vi, när vi träffar en ny människa, prediktera dennes lön baserat på dennes ålder. Denna förmåga, att datorn lärt sig ett samband mellan lön och ålder kan sättas i kontrast till traditionell programmering där vi explicit instruerar datorn vad den ska göra i olika situationer. Exempelvis genom konditionell logik såsom if-satser.

Den andra definitionen betonar att det är såväl en vetenskap som konst. Det innebär att när vi till exempel ska skapa en ML-modell så finns det rätta svar på frågor som “hur funkar en modell rent matematiskt” eller “hur funkar modellens träningsalgoritm”. Detta är vetenskapsdelen. Däremot finns det inte universella rätta svar på frågor såsom “är modellen lämplig att använda” eller “hur ska vi hantera diverse utmaningar som uppstår i modelleringen?”. Detta är konstdelen. Notera, även om det inte finns universellt rätta svar så finns det svar som är mer eller mindre väl-underbyggda.

Inom ML finns det i sin tur ett delämne som kallas för djupinlärning (förkortas ofta DL efter engelskans *deep learning*) där modeller som kallas för neurala nätverk används. Neurala nätverk har en modellarkitektur som ursprungligen var inspirerade av hur bland annat människohjärnan fungerar. Vi kommer lära oss mer om neurala nätverk i bokens

tredje del som startar med Kapitel 7. Förhållandet mellan AI, ML och DL är som ses i Figur 1.1.



Figur 1.1: En schematisk bild över förhållandet mellan AI, ML och DL.

Inom ML kan vi definiera fyra olika typer av problemkategorier, dessa är *regression*, *klassificering*, *dimensionsreducering* och *klustring*. I Kapitel 3 - Kapitel 6 kommer respektive kategori gås igenom på djupet. Härnäst kollar vi på vad dessa fyra problemkategorier innebär.

1.2 Fyra problemkategorier inom maskininlärning: regression, klassificering, dimensionsreducering och klustring

För att träna/skapa maskininlärningsmodeller behöver vi ha tillgång till data. Många gånger har företag och/eller människor tränat modeller som sedan andra kan använda utan att ha direkt tillgång till data. Detta är exempelvis fallet med publikt tillgängliga chatbottar såsom ChatGPT. Men även den modellen har av upphovs-företaget, "OpenAI" i detta fallet, ursprungligen blivit tränad på data.

Data kan generellt sett delas in i *indata* och *utdata*. Indata är den information som ML-modellen får tillgång till och utdata är det svar som ML-modellen förväntas ge efter att ha bearbetat indatan. Generellt sett betecknas utdatan med y och indatan med x .

Inom *regression* antar utdatan, y , kontinuerliga värden. Exempel på kontinuerliga värden är inkomst då den kan anta alla värden inom ett spann, exempelvis 47 831 kr eller 32 421.5 kr.

Inom *klassificering* antar utdatan, y , diskreta värden. Om det är två möjliga värden kallas det för *binär klassificering* och om det är fler möjliga värden kallas det för *multiklass klassificering*. Binär klassificering hade kunnat vara om vi vill prediktera om någon är en man eller en kvinna. I detta fallet är klasserna {man, kvinna}. Multiklass klassificering hade kunnat vara om vi vill prediktera om ett fordon är en cykel, bil, buss eller övrigt. I detta fall finns det alltså fyra klasser, dessa är {cykel, bil, buss, övrigt}.

Det vi kallat för utdata, y , kallas också för den *beroende variabeln*. De engelska begreppen *target* och *label* är också vanligt förekommande. Det vi kallat för indata, x , kallas också för den *oberoende variabeln*. Det engelska begreppet *feature* är också vanligt förekommande.

Kollar vi på datan nedan och exempelvis hade velat skapa en modell där en persons inkomst predikteras baserat på dennes ålder så är det ett regressionsproblem eftersom utdatan (inkomst) är en kontinuerlig variabel.

```
regression_data = {  
  "Inkomst (y)": [58500, 42000, 34000, 39000],  
  "Ålder (x)": [58, 29, 35, 42]  
}
```

①

```
df_reg = pd.DataFrame(regression_data)
df_reg
```

②

- ① Rådata skapas som kommer konverteras till en *Pandas dataframe* i nästa steg.
- ② Vi konverterar rådatan till en *Pandas dataframe*.

	Inkomst (y)	Ålder (x)
0	58500	58
1	42000	29
2	34000	35
3	39000	42

i Vad är indata (x) respektive utdata (y)?

Generellt sett är det inte givet vad som är indata (x) respektive utdata (y), det är något vi som modellerare behöver ta ställning till där vi betraktar y som beroende av x och inte tvärtom. I exemplet ovan är det relativt naturligt att åldern påverkar inkomst och inte tvärtom. Vi blir inte yngre eller äldre om vi får en ökad inkomst, däremot påverkar åldern inkomsten. Många gånger förstår vi ganska snabbt vad som är utdata respektive indata men det kan också vara utmanande att bestämma vad vi ska betrakta som vad vilket är en del av konsten inom ML.

Önskar vi betona rad och/eller kolumn för datan så kan vi använda indexering. Datan ovan hade då kunnat karakteriseras enligt Ekvation 1.1.

$$\begin{cases} y_1, & x_1 \\ y_2, & x_2 \\ y_3, & x_3 \\ y_4, & x_4 \end{cases} \quad (1.1)$$

Vi ser exempelvis att $y_1 = 58500$ och $x_1 = 58$. Notera, i Python börjar indexering från 0 medan vi för ekvationerna har valt att använda indexering som börjar från 1.

En hel rad, t.ex. $y = 42000$ och $x = 29$, kallas tillsammans för en observation eller en datapunkt. Att det kallas för en datapunkt är eftersom vi kan se det som en vektor, som i sin tur kan representeras som en punkt i ett flerdimensionellt rum.

Rent generellt, om vi antar att vi har n stycken rader och p stycken variabler kan vi karakterisera datan enligt Ekvation 1.2.

$$\begin{cases} y_1, & x_{11}, & x_{12}, & \dots, & x_{1p} \\ y_2, & x_{21}, & x_{22}, & \dots, & x_{2p} \\ \vdots & & & & \\ y_n, & x_{n1}, & x_{n2}, & \dots, & x_{np} \end{cases} \quad (1.2)$$

Kollar vi på datan nedan och exempelvis hade velat skapa en modell där en persons kön predikteras baserat på inkomsten så hade det varit ett klassificeringsproblem eftersom utdatan (kön) antar diskreta värden. Notera att i datan nedan så hade vi kunnat använda såväl inkomst som ålder för att prediktera kön. Generellt sett så är frågan om vilka variabler som ska inkluderas i en modell såväl en vetenskap som en konst. Det finns olika tillvägagångssätt och det är ett viktigt delområde inom ML som kallas för *variabelselektion*. Vi kommer lära oss mer om det i Avsnitt 1.3.7.

```
classification_data = {
    "Kön (y)": ["Man", "Kvinna", "Man", "Kvinna"],
    "Inkomst (x1)": [84000, 36750, 33000, 53000],
    "Ålder (x2)": [58, 29, 35, 42]
}

df_classification = pd.DataFrame(classification_data)
df_classification
```

	Kön (y)	Inkomst (x1)	Ålder (x2)
0	Man	84000	58
1	Kvinna	36750	29
2	Man	33000	35
3	Kvinna	53000	42

i Notera, i datan ovan har vi de två kategorierna “Man” och “Kvinna”. En annan kategorisering hade kunnat vara “Man”, “Kvinna” samt “Vill ej uppge”. Då har vi gått från ett binärt klassificeringsproblem till ett multiklass klassificeringsproblem. Hur gör vi i praktiken? Det är något den som skapar modellerna behöver ta ställning till, bland annat med avseende på vad som vill uppnås.

Inom *dimensionsreducering* förenklar vi indata genom att överföra den till en rymd med lägre dimensionalitet, exempelvis genom en modell som kallas för *principalkomponentanalys*, ofta benämnt PCA efter engelskans *Principal Component Analysis*. En vanlig anledning till att minska datans dimensionalitet är för att modellträningen (där vi använder regressionsmodeller eller klassificeringsmodeller) ska gå snabbare.

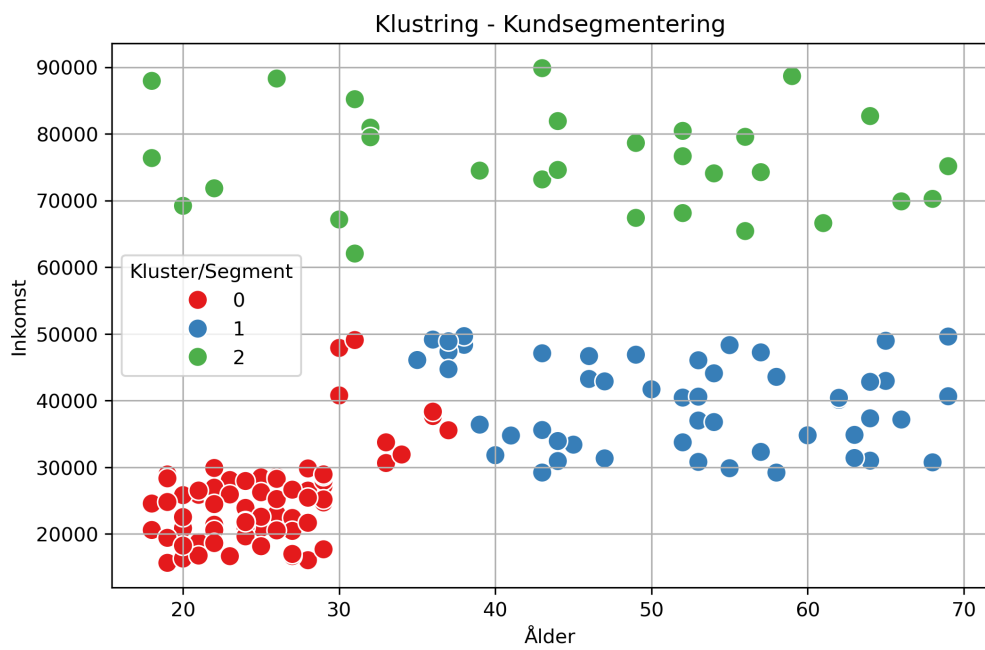
Inom *klustring* har vi endast indata och vill dela in denna i olika grupper/kluster. Ett tillämpningsområde inom detta är kundsegmenteringsmodeller. Exempelvis i Figur 1.2 kan vi tänka oss att kunder från de olika klustren/segmenten kan ha olika preferenser och köpkraft. Vilka preferenser de har kan exempelvis undersökas via intervjuer och/eller via dataanalys om data finns tillgänglig. Detta kan företag använda för att bland annat anpassa produktbudanden och marknadsföring. Om vi tänker oss att ett bilföretag producerar både familjebilar och sportbilar så kanske sportbilarna hade marknadsförts mot det andra segmentet eftersom de har högst inkomst. Vi kanske också kommer fram till att många inom det första segmentet hade önskat ha en sportbil, då kanske det kan vara en bra affärsmöjlighet för företaget att försöka utveckla en billigare form av sportbil?

Vi avslutar med att nämna att ett gemensamt namn för regressionsproblem och klassificeringsproblem är *väglett lärande* (eller övervakat lärande) och ett gemensamt namn för klustring och dimensionsreducering är *icke-väglett lärande* (eller oövervakat lärande). Anledningen till att det kallas för väglett lärande är på grund av att vi kan tänka oss att y vägleder x när modellerna tränas. För icke-väglett lärande använder vi endast indatan, x , innebärande att inget y vägleder träningen.

1.3 Fundamentala koncept

Inom maskininlärning finns det ett antal olika koncept som är fundamentala och väldigt ofta används. Nedan listar vi dessa.

- Uppdelning av data i träningsdata, valideringsdata och testdata. En variant av detta är k-delad korsvalidering som vi också kommer lära oss.



Figur 1.2: Exempel på en kundsegmenteringsmodell.

- Utvärderingsmått, för regressionsproblem är det vanligast förekommande det som kallas för *Root Mean Squared Error (RMSE)*. För klassificeringsproblem finns det andra utvärderingsmått såsom *accuracy*, *precision*, *recall* och *confusion matrix*. Här kommer vi introducera *RMSE* och andra utvärderingsmått presenteras senare i boken.
- Hyperparametrar och parametrar. Hyperparametrar styr hur en modell lär sig och svarar på frågan “hur vi lär oss”. Parametrar är saker som en modell har lärt sig och svarar på frågan “vad har vi lärt oss”.
- *Grid search* används för att optimera hyperparametrar.
- Hantering av kategorisk data i modeller, det vill säga data som är icke-numerisk. Om vi antar att en variabel kan anta färgerna {blå, grön, röd} så är det exempel på kategorisk data.
- *Feature engineering* handlar om vilka variabler som ska väljas i en modell, skapandet av nya variabler och transformering av variabler. Generellt sett åsyftas de oberoende variablerna (*features*) men även exempelvis transformering av de beroende variablerna kan göras.

Vi går nu igenom varje koncept i mer detalj.

1.3.1 Träningsdata, valideringsdata och testdata

När vi tränar ML-modeller så behöver vi data som modellerna kan tränas på. Mer specifikt brukar datan, det vill säga hela datasetet, delas in i träningsdata, valideringsdata och testdata.

- På träningsdatan tränar vi olika modeller.
- På valideringsdatan utvärderar vi de olika modellerna som tränats på träningsdatan för att se vilken modell som presterar bäst. Hur vet vi vilken modell som presterar bäst? För att veta det använder vi ett eller flera utvärderingsmått. För regressionsproblem används ofta *RMSE* som vi snart kommer lära oss om i Avsnitt 1.3.3.
- När vi valt den bäst presterande modellen från valideringsdatan ska vi, givet att vi är nöjda med resultatet, träna om den modellen på träningsdatan och valideringsdatan ihopslagen innan modellen slutligen testas på testdatan för att få en uppskattning på modellens generaliseringsförmåga på ny osedd data. Om vi inte är nöjda med resultatet får vi exempelvis prova att träna andra modeller på träningsdatan, bearbeta datan på olika sätt eller samla in ny/mer data.

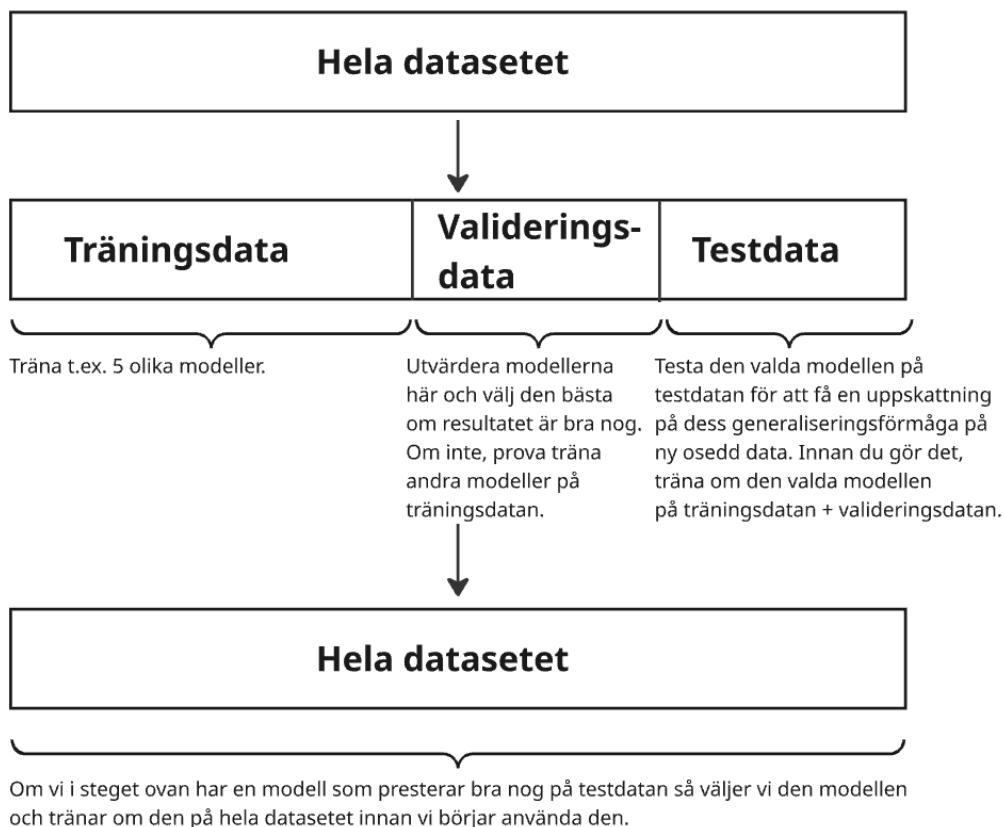
- Om modellen som testats på testdata bedöms vara tillräckligt bra för att börja användas/produktionssättas så ska modellen tränas om på hela datasetet, det vill säga träningsdata, valideringsdata och testdata ihopslagen. Det är också anledningen till att vi ska träna om modellen på träningsdata och valideringsdata i steget ovan innan vi testar den på testdata. Detta eftersom det är närmre mängden data som modellen slutligen kommer tränas på innan produktionssättning och därmed får vi generellt sett också en mer precis bild av hur bra modellen kan tänkas prestera i skarpt läge när den är i produktion.

i I praktiken ser vi ibland att en modell inte tränas om på träningsdata och valideringsdata ihopslagen innan den testas på testdata eller att modellen inte tränas om på hela datasetet innan den börjar användas skarpt. Anledningen är att det kan ta lång tid att träna om modellen och den som genomför modelleringen tycker helt enkelt inte att det är värt det. Så länge man förstår vad man gör så kan man avvika från det som är teoretiskt korrekt.

En tumregel är att data används i proportionerna 60-20-20 eller 70-15-15, det vill säga 60% (70%) av data används för träning, 20% (15%) av data används för validering och 20% (15%) av data används för test. Denna uppdelning är endast en tumregel. Om vi exempelvis har för lite träningsdata kan träningen av modellerna bli dålig och har vi för lite valideringsdata kan valet av den bäst presterande modellen bli dålig. Det finns alltså en *trade-off* men denna *trade-off* beror på den *absoluta* mängden data snarare än procentsatser. Det innebär att om vi har mycket data (vad som är mycket data beror på tillämpningsområde och modell) kan det sakna praktisk betydelse om vi använder en 60-20-20 uppdelning, 50-25-25 uppdelning eller ens en 98-1-1 uppdelning. Om modellerna tar lång tid att träna kan vi föredra en uppdelning där vi har mindre data i träningsdata på grund av att modellerna då tränas på mindre data vilket går snabbare. Om vi har lite data kanske vi väljer att använda en uppdelning där vi har mer data i träningsdata för att modellerna ska ha en rimlig chans att bli tränade på ett tillräckligt stort dataset.

Se Figur 1.3 för en schematisk illustration av hur vi använder träningsdata, valideringsdata och testdata.

Varför exakt behöver vi använda valideringsdata? Anledningen är att vi på träningsdata kan skapa perfekta modeller som kan genomföra alla prediktioner korrekt. Detta är dock till ingen nytta om modellen inte kan prediktera ny, osedd data korrekt. Därför utvärderar vi modellerna som tränats på träningsdata via valideringsdata. Modeller som är bra på träningsdata men dåliga på valideringsdata sägs vara överanpassade



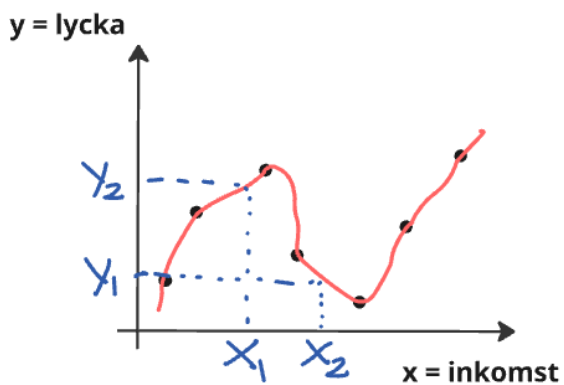
Figur 1.3: En schematisk illustration av hur träningsdata, valideringsdata och testdata används.

(*overfitted* på engelska). Modeller som är för enkla för att kunna fånga strukturen i träningsdatan sägs vara underanpassade (*underfitted* på engelska). Generellt sett presterar underanpassade modeller dåligt på både träningsdatan och valideringsdatan. Se Figur 1.4 för ett schematiskt exempel på hur en överanpassad modell, underanpassad modell och en rimlig modell hade kunnat se ut. I figuren antar vi att den beroende variabeln, y , mäter en persons lycka och den oberoende variabeln, x , mäter en persons inkomst. Generellt sett är det rimligt att tänka sig att en ökad inkomst ger en ökad lycka. Kollar vi på det översta exemplet “a) överanpassad modell” så ser vi att modellen, som representeras av den röda linjen följer träningsdatan, som representeras av de svarta punkterna, exakt. Det leder till orimliga resultat. Anledningen är att om två personer har de fixa inkomstnivåerna x_1 och x_2 där $x_1 < x_2$ i figuren så blir den predikterade lyckan $y_2 > y_1$. I ord, personen med lägre inkomst förväntas ha en högre lycka i just det intervallet vi kollar på. Det är ett orimligt mönster. Modellen är alltså perfekt för träningsdatan men leder till orimliga resultat när vi genomför nya prediktioner. Den är överanpassad. Kollar vi på det mittersta exemplet “b) underanpassad modell” så ser vi en underanpassad modell som inte lyckas fånga datans underliggande struktur. Oavsett inkomstnivå så är den predikterade lyckan densamma. I det nedersta exemplet, “c) rimlig modell” så ser vi en modell som verkar följa den generella trenden i träningsdatan utan att följa den för noga. Vi ser också att om två personer med de fixa inkomsterna x_3 och x_4 där $x_3 < x_4$ så förväntas personen med den högre inkomsten ha en högre lycka y_4 än personen med lägre inkomst som förväntas ha lyckan y_3 . Det är rimligt ur ett modelleringsperspektiv även om det i verkligheten kan finnas många andra faktorer som påverkar lyckan. Detta är alltså ett förenklat exempel.

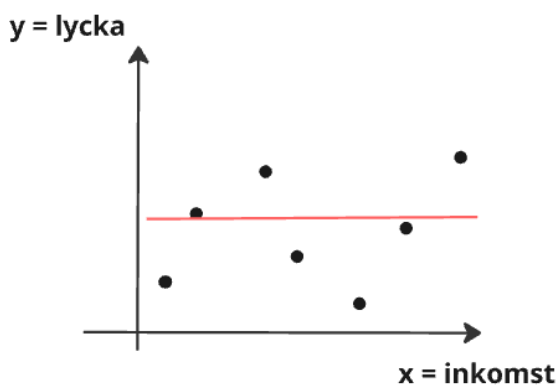
i En modell är en förenkling av verkligheten

I exemplet ovan skapade vi en modell där lycka modellerades med avseende på inkomst. Det är en förenkling av verkligheten eftersom vi vet att många andra faktorer såsom relationer och hälsa också påverkar lycka. Faktum är att alla modeller är en förenkling av verkligheten, de varken är eller bör betraktas som “sanna”. Målet är alltså inte att skapa en modell som ger en “exakt bild av verkligheten” utan snarare en modell som hjälper oss att uppnå det syfte vi har. Därför kan en viss modell vara användbar i ett sammanhang men inte i ett annat.

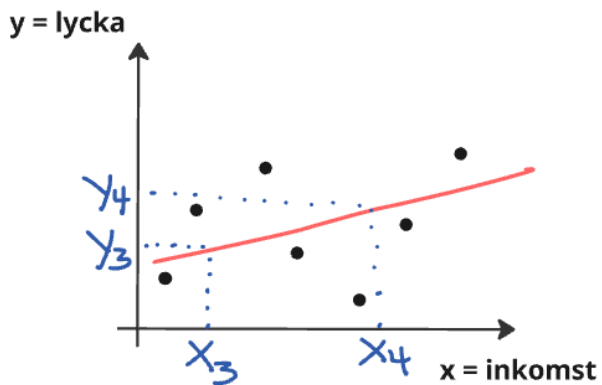
Varför testar vi en modell på testdatan? Räcker det inte att modellen utvärderas på valideringsdatan? Det faktum att man inom ML ofta skapar flera olika modeller och väljer den modell som presterar bäst på valideringsdatan gör att det av ren slump kan hända att modellen råkar prestera bra på valideringsdatan. För att minska denna risk



a) Överanpassad modell



b) Underanpassad modell



c) Rimlig modell

Figur 1.4: En schematisk figur som visar hur en överanpassad modell, underanpassad modell och en rimlig modell hade kunnat se ut.

så genomförs alltså ett test på testdatan. Notera att testdatan tar man ut i början av sitt arbetsflöde och därefter ska man inte använda det förrän man kommit till slutet av arbetsflödet och det är dags att slutgiltigt testa den valda modellen. Anledningen är att om vi testar olika modeller på testdatan och fortsätter att göra så tills vi hittar något som funkar bra så finns det en risk att vi till slut endast hittat en modell som presterar bra på den specifika testdatan som vi har. Det är meningslöst eftersom syftet med testdatan är att testa en modells generaliseringsförmåga på ny, osedd data.

Notera att idealt sett så önskar vi att datan som används är *representativ* för det vi vill modellera. Om vi exempelvis vill skapa en prediktionsmodell för att prediktera inkomster för personer med olika utbildningsnivå i Sverige så vore det inte bra om vi endast har data för personer som bor i Stockholm. Anledningen kan exempelvis vara att utbildningsnivå har olika stor påverkan på inkomst i olika delar av landet.

Har vi ett representativt urval av data i grunden så gör *scikit-learn* per automatik slumpmässiga urval när data delas in i träningsdata, valideringsdata och testdata (i kodexemplet nedan kommer vi se `train_test_split`). Det slumpmässiga urvalet leder oftast till att även träningsdatan, valideringsdatan och testdatan blir representativ. Vi nämner för den intresserade läsaren att det är möjligt att genomföra *stratifierade urval* när data delas in i träningsdata, valideringsdata och testdata. Detta är inget vi går djupare in på i denna bok.

Vi kollar nu på ett kodexempel för att konkretisera det vi gått igenom. Vi använder demonstrationsdatan `load_diabetes` från *scikit-learn* för detta ändamål.

```
from sklearn.datasets import load_diabetes
from sklearn.model_selection import train_test_split ①

diabetes = load_diabetes(as_frame=True) ②

df = diabetes.frame ③
print(df.head()) ④

X = df.drop(columns='target')
y = df['target'] ⑤

X_train_full, X_test, y_train_full, y_test = train_test_split(X,
    ↪ y, test_size=0.2, random_state=42) ⑥
```

```
X_train, X_val, y_train, y_val = train_test_split(X_train_full,
↪ y_train_full, test_size=0.25, random_state=42) ⑦
```

- ① Importerar de två biblioteken vi använder.
- ② Ladda in datan.
- ③ `.frame` specificerar att vi vill ha en *dataframe* som innehåller både de oberoende variablerna (x) och den beroende variabeln (y) som på engelska benämns *target*.
- ④ Skriver ut de fem första raderna så vi ser hur datan ser ut.
- ⑤ Vi delar in vår data i X och y för att därefter, i nästa steg, dela upp vår data i träningsdata, valideringsdata och testdata.
- ⑥ Vi delar in vår data, X och y , i `X_train_full`, `X_test`, `y_train_full` och `y_test`. `random_state=42` leder till att vi får reproducerbara resultat eftersom vilken data som hamnar i `train_full` respektive `test` är slumpmässigt. Siffran 42 valdes godtyckligt.
- ⑦ `X_train_full` och `y_train_full` delas nu in i `X_train`, `X_val`, `y_train` och `y_val`.

	age	sex	bmi	bp	s1	s2	s3
0	0.038076	0.050680	0.061696	0.021872	-0.044223	-0.034821	-0.043401
1	-0.001882	-0.044642	-0.051474	-0.026328	-0.008449	-0.019163	0.074412
2	0.085299	0.050680	0.044451	-0.005670	-0.045599	-0.034194	-0.032356
3	-0.089063	-0.044642	-0.011595	-0.036656	0.012191	0.024991	-0.036038
4	0.005383	-0.044642	-0.036385	0.021872	0.003935	0.015596	0.008142

	s4	s5	s6	target
0	-0.002592	0.019907	-0.017646	151.0
1	-0.039493	-0.068332	-0.092204	75.0
2	-0.002592	0.002861	-0.025930	141.0
3	0.034309	0.022688	-0.009362	206.0
4	-0.002592	-0.031988	-0.046641	135.0

Sammanfattningsvis har vi med koden ovan skapat följande:

1. Vi har hela datasetet som representeras av `X` och `y`.
2. Vi har träningsdatan och valideringsdatan ihopslagen som representeras av `X_train_full` och `y_train_full`.
3. Vi har träningsdatan som representeras av `X_train` och `y_train`.
4. Vi har valideringsdatan som representeras av `X_val` och `y_val`.
5. Vi har testdatan som representeras av `X_test` och `y_test`.

Med datan på plats kan vi demonstrera hur processen ser ut vid modellval. Kom ihåg att processen var:

1. Träna olika modeller på träningsdatan.
2. Utvärdera modellerna på valideringsdatan och välj den bäst presterande.
3. Om prestationen är bra nog, träna om den bäst presterande modellen på träningsdatan och valideringsdatan ihopslagen innan den utvärderas på testdatan.
4. Träna om modellen på hela datasetet innan vi börjar använda den skarpt.

För detta ändamål skapar vi två modeller, en linjär regressionsmodell och ett besluts-träd, som vi kommer lära oss mer om senare i boken, Kapitel 3.

```
from sklearn.linear_model import LinearRegression
from sklearn.tree import DecisionTreeRegressor
from sklearn.metrics import root_mean_squared_error

# 1. Träna två modeller på träningsdatan.
linreg = LinearRegression() ①
tree = DecisionTreeRegressor(random_state=42) ②

linreg.fit(X_train, y_train)
tree.fit(X_train, y_train) ③

# 2. Utvärdera modellerna på valideringsdatan och välj den bäst
    ↪ presterande.
rmse_linreg = root_mean_squared_error(y_val,
    ↪ linreg.predict(X_val))
rmse_tree = root_mean_squared_error(y_val, tree.predict(X_val)) ④

print(f"RMSE på validering - Linjär Regression:
    ↪ {rmse_linreg:.2f}")
print(f"RMSE på validering - Beslutsträd: {rmse_tree:.2f}")

# Välj den bäst presterande modellen på valideringsdatan. Den
    ↪ modellen som har lägre RMSE är bättre.
if rmse_linreg < rmse_tree:
    best_model = LinearRegression()
    best_model_name = "Linjär Regression"
else:
```

```

best_model = DecisionTreeRegressor(random_state=42)
best_model_name = "Beslutsträd"

# 3. Om prestationen är bra nog, träna om den bäst presterande
↪ modellen på träningsdatan och valideringsdatan ihopslagen
↪ innan den utvärderas på testdatan.
best_model.fit(X_train_full, y_train_full)

# Utvärdera modellen på testdatan.
rmse_test = root_mean_squared_error(y_test,
↪ best_model.predict(X_test))

print(f"Bästa modell baserat på validering: {best_model_name}")
print(f"RMSE på testdatan: {rmse_test:.2f}")

# 4. Träna om modellen på hela datasetet innan vi börjar använda
↪ den skarpt.
best_model.fit(X, y)

```

- ① Vi instantierar en linjär regressionsmodell.
- ② Vi instantierar ett beslutsträd.
- ③ Vi tränar respektive modell genom att använda `.fit()` metoden.
- ④ Vi beräknar *RMSE* för respektive modell. Vad det utvärderingsmåttet innebär kommer vi gå igenom i Avsnitt 1.3.3.

RMSE på validering - Linjär Regression: 51.19

RMSE på validering - Beslutsträd: 65.81

Bästa modell baserat på validering: Linjär Regression

RMSE på testdatan: 53.85

fit_intercept	True
copy_X	True
tol	1e-06
n_jobs	None
positive	False

Att dela in datan i träningsdata, valideringsdata och testdata är fundamentalt och

kommer återkomma genom hela boken. En variant av detta är det som kallas för k-delad korsvalidering och det går vi igenom härnäst.

1.3.2 K-delad korsvalidering

Som ett alternativ till uppdelningen i träningsdata, valideringsdata och testdata så används ofta *k-delad korsvalidering*. Det är en metod där datan först delas in i träningsdata och testdata. En vanlig tumregel är en 80-20 uppdelning eller 70-30 uppdelning. Träningsdatan delas därefter upp i k lika stora delar där modellen tränas k gånger, varje gång på k-1 delar (dessa k-1 delarna är den nya träningsdatan för varje iteration) och den återstående delen är valideringsdatan för varje iteration. K är vanligtvis fem eller tio. Se Figur 1.5 för en visualisering av k-delad korsvalidering. För varje iteration fås ett värde på det utvärderingsmåttet vi använder, exempelvis *RMSE* om vi arbetar med regressionsproblem. Om vi använder 5-delad korsvalidering, som i figuren, kommer vi totalt få fem olika siffror som vi tar medelvärdet av vilket gör att vi får en slutgiltig siffra som används för att mäta hur bra vår modell presterar.

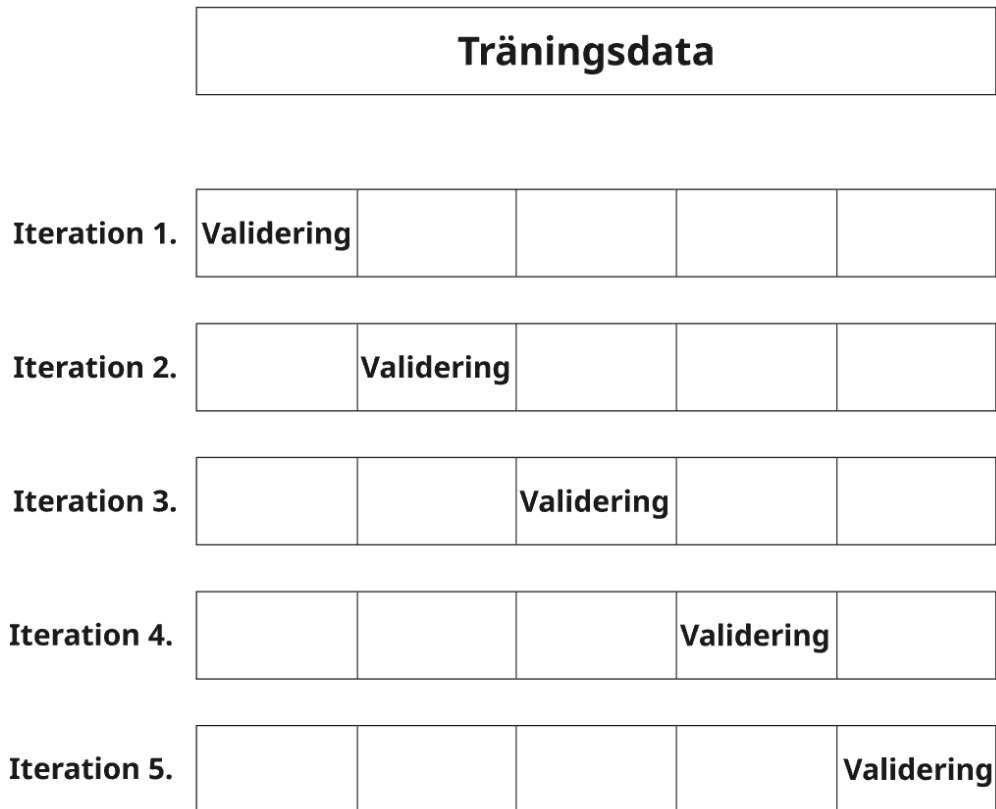
Logiken bakom att använda k-delad korsvalidering är att om valideringsdatan är för liten så kan det leda till ett dåligt modellval. Om valideringsdatan är för stor kan det leda till lite träningsdata vilket kan leda till att modellerna blir dåligt tränade. Men även i de fall valideringsdatan varken är för liten eller för stor så kombinerar k-delad korsvalidering flera estimat på utvärderingsmåttet som används (t.ex. *RMSE* i regressionsfallet) vilket kan ge en bättre skattning på hur bra modellen faktiskt presterar. Nackdelen är att varje modell behöver tränas k gånger, något som kan ta lång tid.

Vi demonstrerar K-delad korsvalidering med ett kodexempel.

```
import numpy as np
from sklearn.datasets import load_diabetes
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.svm import LinearSVR
from sklearn.model_selection import cross_validate

X, y = load_diabetes(return_X_y=True)
X_train, X_test, y_train, y_test = train_test_split(X, y,
    ↪ test_size=0.2, random_state=42)

lr = LinearRegression()
```

Figur 1.5: Illustration över hur en 5-delad korsvalidering ser ut. I varje iteration så genomförs en validering på den data som representeras av rutan där det står "Validering". Datan som representeras med övriga rutor används för att träna modellen. För varje iteration fås ett värde för det utvärderingsmått som används på valideringen, exempelvis RMSE för regressionsproblemet. Totalt fås fem värden som används för att beräkna ett medelvärde, medelvärdet blir den siffra som används för att utvärdera hur bra modellen presterat.

```

svm = LinearSVR() ①

cv_lr = cross_validate(lr, X, y, cv=5, scoring =
    ↪ 'neg_root_mean_squared_error') ②
cv_svm = cross_validate(svm, X, y, cv=3, scoring =
    ↪ 'neg_root_mean_squared_error')

print(cv_lr.keys()) ③

print(cv_lr['test_score']) ④
print(cv_svm['test_score']) ⑤

if np.mean(cv_lr['test_score']) > np.mean(cv_svm['test_score']):
    ↪ ⑥
    print("The best model is linear regression.")
elif np.mean(cv_lr['test_score']) ==
    ↪ np.mean(cv_svm['test_score']):
    print("The models are equally good.")
else:
    print("The best model is support vector machine.")

```

- ① Vi instantierar den linjära regressionsmodellen och en modell som heter *Support Vector Machine (SVM)* som vi kommer lära oss om senare i boken, Kapitel 3.
- ② Vi genomför 5-delad korsvalidering. I kodraden nedan genomför vi 3-delad korsvalidering så läsaren ska kunna se skillnaden. Vi specificerar hyperparametern `scoring = 'neg_root_mean_squared_error'`, i Avsnitt 1.3.5 kommer det förklaras varför vi använder *negative root mean squared error*. I korthet så vill vi generellt sett ha så lågt fel som möjligt, det vill säga låg *RMSE*. I *scikit-learn* är dock “högre värden bättre” och för att rangordningen då ska bli korrekt använder vi negativt tecken.
- ③ Med denna koden ser vi att vi bland annat kan få ut `test_score` vilket är det vi är intresserade av i detta kodexempel.
- ④ Vi får fem estimerade fel eftersom vi använde 5-delad korsvalidering vilket specificerades med `cv=5` i koden ovan.
- ⑤ Vi får tre estimerade fel eftersom vi använde 3-delad korsvalidering vilket specificerades med `cv=3` i koden ovan.
- ⑥ Med korsvalidering fick vi flera estimat på felen som modellerna gör. För att jämföra dem beräknar vi medelvärdet av felen för respektive modell. Notera, vi vill

generellt sett ha låga fel men eftersom “högre värden är bättre” i *scikit-learn* så använde vi `neg_root_mean_squared_error` och då är alltså den modell med högre värde bättre.

```
dict_keys(['fit_time', 'score_time', 'test_score'])
[-52.72497937 -55.03486476 -56.90068179 -54.85204179 -53.94638716]
[-90.2051448 -93.11857727 -94.32004223]
The best model is linear regression.
```

1.3.3 RMSE

Root Mean Squared Error (RMSE) är ett utvärderingsmått för regressionsmodeller och dess matematiska uttryck ser vi i Ekvation 1.3.

$$\text{RMSE} = \sqrt{\frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2} \quad (1.3)$$

Notera att $\hat{y}_i$ är predikterade värden för observation/datapunkt i , och y_i är de sanna värdena för observation/datapunkt i . Det är en väletablerad konvention att tillägget av en “hatt”, $\hat{y}$, generellt sett betecknar predikterade värden.

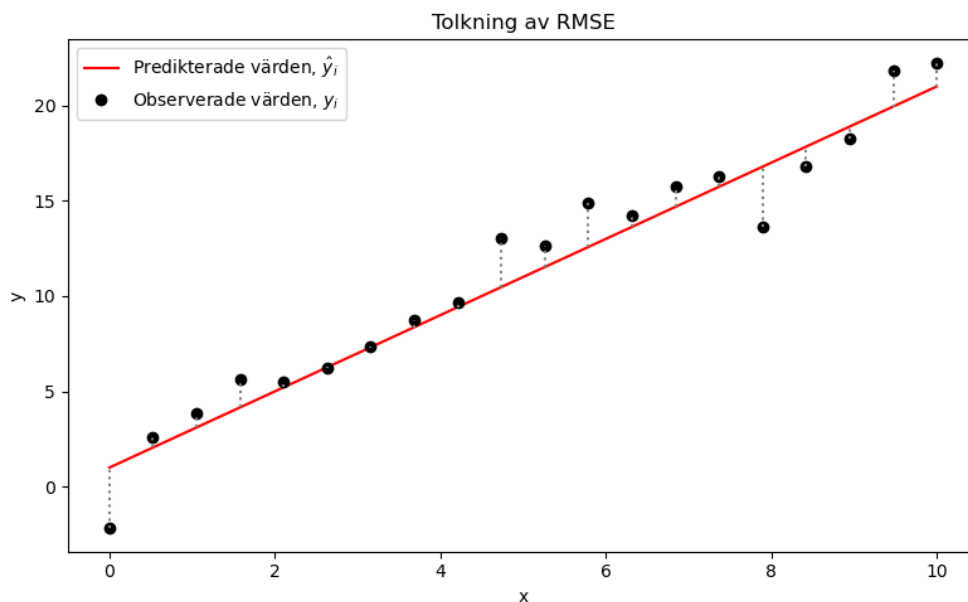
Vi går nu igenom formeln steg för steg.

- Vi beräknar skillnaden mellan det sanna värdet, y_i och det predikterade värdet $\hat{y}_i$; $(y_i - \hat{y}_i)$. Detta kallas för *Error*.
- Eftersom vi beräknar skillnader kan dessa vara positiva och negativa vilket i sin tur kan leda till att skillnaderna tar ut varandra när de summeras. Antag att $y_1 = 10, y_2 = 8$ och $\hat{y}_1 = 12, \hat{y}_2 = 6$. Då blir summan av skillnaderna $(y_1 - \hat{y}_1) + (y_2 - \hat{y}_2) = (10 - 12) + (8 - 6) = (-2) + (2) = 0$. Summan av skillnaderna blir alltså noll trots att båda prediktionerna är fel. Detta är inte en önskvärd egenskap och därför kvadrerar vi skillnaderna och tar alltså $(y_i - \hat{y}_i)^2$. Detta kallas för *Squared Error*.
- Vi beräknar medelvärdet för de kvadrerade felen (*Squared Error*); $\frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2$. Detta kallas för *Mean Squared Error*.
- När vi beräknade *Squared Error* så kvadrerade vi skillnaderna. Om vi exempelvis predikterar en persons lön i svenska kronor (kr) så blir enheten $(kr)^2$ vilket är svårtolkat. Därför använder vi kvadratroten för att återföra värdena till den

ursprungliga enheten och får formeln $\sqrt{\frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2}$. Detta kallas för *Root Mean Squared Error*.

Vi ser alltså att namnet *Root Mean Squared Error* är ett beskrivande namn på formeln.

Vi kan, lite slarvigt, tolka *RMSE* som de observerade värdenas (y) medelavstånd från de predikterade värdena ($\hat{y}$). Att det är en slarvig tolkning beror på att vi tar medelvärdet av *Squared Error* och därefter tar vi kvadratroten ur det. Se Figur 1.6.



Figur 1.6: En visualisering som hjälper oss att tolka RMSE.

Avslutningsvis, vill vi kunna tolka felet som modeller gör använder vi oss av *RMSE*. Vill vi rangordna modeller, det vill säga jämföra dem för att se vilken som är bättre kan vi också använda oss av *RMSE* men vi kan även använda *Mean Squared Error* (*MSE*). Anledningen är att det endast skiljer ett roten-ur tecken och det är en monoton transformation innebärande att rangordningen förblir densamma. Exempelvis, $100 > 9 \Leftrightarrow \sqrt{100} > \sqrt{9} \Leftrightarrow 10 > 3$. Att man ofta använder *MSE* för modell-rangordning, som vi bland annat kommer se senare i boken, beror på att kvadratroten är en extra operation som kan ta längre tid att genomföra.

Vi demonstrerar hur beräkningen av *RMSE* kan se ut i *scikit-learn*.

```
from sklearn.metrics import root_mean_squared_error
y_true = [4, 10, 8]
y_pred = [2, 13, 5]
root_mean_squared_error(y_true, y_pred)
```

①

②

③

- ① Vi antar att vi har några specificerade värden som är korrekta (y).
- ② Vi antar att vi har några värden som motsvarar predikterade värden ($\hat{y}$).
- ③ Vi beräknar *RMSE*.

2.70801280154532

1.3.4 Hyperparametrar och parametrar

En hyperparameter svarar på frågan “hur vi lär oss” och en parameter svarar på frågan “vad vi lärt oss”.

I koden nedan har vi skapat två olika modeller, `lin_reg_1` och `lin_reg_2`. Notera att båda modellerna är linjära regressionsmodeller vars modellspecifikation är den räta linjens ekvation i detta fallet, det vill säga $y = kx + m$ där k är lutningen och m är interceptet eller skärningspunkten med y-axeln. Skillnaden är att i den första modellen är hyperparametern `set_intercept` satt till `False` medan i den andra modellen är hyperparametern `set_intercept` satt till `True`. Detta styr alltså hur vi lär oss och i den första modellen specificerar vi att vi inte vill att modellen ska skatta ett intercept utifrån datan och istället anta att det är 0 medan i den andra modellen vill vi att modellen ska skatta ett intercept utifrån datan. Därmed kommer interceptet, som är en parameter, att vara 0 för den första modellen och något annat för den andra modellen (teoretiskt sett kan interceptet bli 0 även för den andra modellen ifall det är vad modellen lär sig).

```
from sklearn.linear_model import LinearRegression

# Data
example_data = {
    "y": [-10, 3, 2, 7, -9, 10, 9, 7, 4, 7, 3, 2, -4, -5, -9, -4],
    "x": [-3, 0, 1, 2, 4, 2, 3, 5, 7, 8, 9, 6, -2, -5, -1, -2]
}
```

①

```

example_data = pd.DataFrame(example_data) ②

lin_reg_1 = LinearRegression(fit_intercept=False) ③
lin_reg_1.fit(example_data[["x"]], example_data["y"]) ④

lin_reg_2 = LinearRegression(fit_intercept=True)
lin_reg_2.fit(example_data[["x"]], example_data["y"]) ⑤

print('Parameters for lin_reg_1.', 'Intercept:',
      ↪ round(lin_reg_1.intercept_, 2), 'Slope',
      ↪ round(lin_reg_1.coef_[0], 2)) ⑥
print('Parameters for lin_reg_2.', 'Intercept:',
      ↪ round(lin_reg_2.intercept_, 2), 'Slope',
      ↪ round(lin_reg_2.coef_[0], 2)) ⑦

```

- ① Importera den linjära regressionsmodellen från *scikit-learn* biblioteket.
- ② Skapa ett dataset som vi kommer använda för demonstrationssyfte.
- ③ Instantiera en linjär regressionsmodell där hyperparametern `fit_intercept` är satt till `False`.
- ④ Träna modellen på den skapade exempeldata.
- ⑤ En linjär regressionsmodell där hyperparametern `fit_intercept` är satt till `True` instantieras och tränas.
- ⑥ Vi skriver ut interceptet (m i den räta linjens ekvation $y = kx + m$) och lutningen (k i den räta linjens ekvation $y = kx + m$) för `lin_reg_1` modellen.
- ⑦ Vi skriver ut interceptet och lutningen för `lin_reg_2` modellen.

```

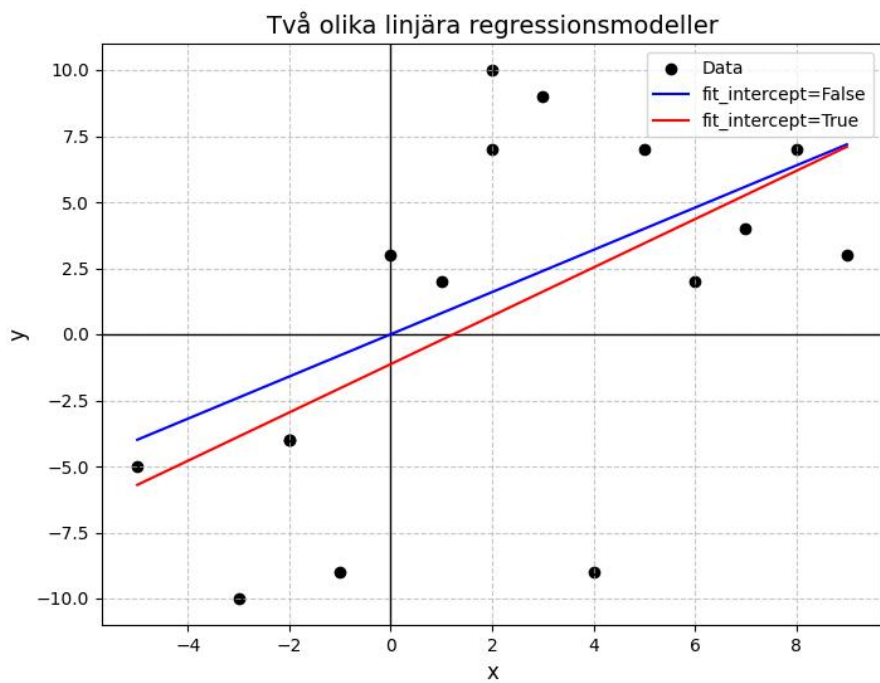
Parameters for lin_reg_1. Intercept: 0.0 Slope 0.8
Parameters for lin_reg_2. Intercept: -1.13 Slope 0.91

```

Från resultatet av koden ser vi att den första modellen har interceptet 0 och lutningen 0.8 medan den andra modellen har interceptet -1.13 och lutningen 0.91. Vi kan visualisera de två modellerna vi skapat. Se Figur 1.7.

1.3.5 Grid search

Ovan provade vi manuellt två olika värden på en hyperparameter, `fit_intercept=False` och `fit_intercept=True`. Senare i boken kommer vi se att vi kommer kunna välja bland många olika värden för flera olika typer av hyperparametrar och det vore krångligt att behöva göra det manuellt då det snabbt kan bli väldigt mycket kod. Vad



Figur 1.7: Visualisering av två olika linjära regressionsmodeller.

vi istället använder, och detta kommer vi frekvent använda under bokens gång, kallas för `GridSearch` i *scikit-learn*. Detta är ett automatiserat sätt att välja hyperparametrar på. Vad algoritmen gör är att den för varje unik kombination av hyperparametrar tillämpar k-delad korsvalidering och ser vilken kombination som är optimal.

I koden nedan undersöker vi endast en hyperparameter `fit_intercept` som antar värdet `False` eller `True`.

```
from sklearn.model_selection import GridSearchCV ①

model = LinearRegression() ②

hyperparameters = {'fit_intercept': [True, False]} ③

grid_search = GridSearchCV(estimator=model,
    ↪ param_grid=hyperparameters, scoring='neg_mean_squared_error',
    ↪ cv=2) ④

grid_search.fit(example_data[["x"]], example_data[["y"]]) ⑤
```

- ① Importera `GridSearchCV` från *scikit-learn* biblioteket.
- ② Instantiera en linjär regressionsmodell.
- ③ Här specificerar vi vilka hyperparametrar vi vill undersöka. I detta fallet är det hyperparametern `fit_intercept` där vi vill prova värdena `True` och `False`. Senare i boken kommer vi för mer komplexa modeller se att vi kan undersöka flera kombinationer av hyperparametrar.
- ④ Vi instantierar en `GridSearch`. Notera att hyperparametern `cv` har i *scikit-learn* standardvärdet fem men vi har valt värdet två. Anledningen är eftersom vi senare kommer att skriva ut resultaten och vill kunna få plats med allt på bokens sidor. Vi specificerade även `'neg_mean_squared_error'`, förklaringen till varför vi använder negativa värden på *MSE* ges i informationsrutan nedan med rubriken “Högre är bättre i *scikit-learn scoring*”.
- ⑤ Vi genomför en *grid search* genom att använda `.fit()` metoden. Vi använder `example_data` från kodexemplet i Avsnitt 1.3.4.

i Högre är bättre i *scikit-learn* scoring

Notera att i kodexemplet ovan användes `scoring='neg_mean_squared_error'` i den fjärde annoteringen där “neg” står för *negative*. Varför *negative*? Anledningen är att i *scikit-learn* är det en allmän princip att “högre värden är bättre” i samband med att *score* beräknas. Det förstås lättast med ett exempel.

Antag att vi har två modeller och att modell 1 har en *MSE* på 100 och modell 2 har en *MSE* på 150. Då är modell 1 bättre eftersom felet (*MSE*) är lägre. Men i *scikit-learn* hade modell 2 betraktats som bättre eftersom *scikit-learn* följer den allmänna principen att “högre värden är bättre” i samband med *scoring* vilket det uppenbart inte är i detta fallet. För att hantera detta tar vi de negativa värdena (*negative mean squared error*) och får -100 och -150. Nu är -100 större än -150 eftersom det är närmre noll. *Scikit-learn* hade därmed betraktat modell 1 som bättre. Nu blir det alltså korrekt rangordning av modellerna!

Vi kan inspektera resultaten av den GridSearch vi genomfört. Vi kommer kolla på några kolumner i taget eftersom samtliga kolumner av utrymmesskäl inte får plats på bokens sida.

i Notera att det finns en viss slumpmässighet i GridSearch eftersom vilken data som inkluderas i korsvalideringen som genomförs inte är deterministiskt. Det innebär att läsaren lär få andra siffror och eventuellt även en annan modell-rangordning.

```
df_gs_results = pd.DataFrame(grid_search.cv_results_) ①
df_gs_results.iloc[:, [0, 1, 5]] ②
```

- ① Spara resultaten från den *grid search* vi genomfört i en *pandas dataframe* och döpa variabeln till `df_gs_results`.
- ② Skriv ut den första, andra och sjätte kolumnen från variabeln `df_gs_results`. Kom ihåg att indexering i Python börjar från 0.

	mean_fit_time	std_fit_time	params
0	0.000502	0.000502	{'fit_intercept': True}
1	0.001019	0.000020	{'fit_intercept': False}

Från tabellen ovan ser vi medelvärde och standardavvikelsen för träningstiden för de två modellerna där den ena hade `fit_intercept = True` och den andra hade `fit_intercept = False`. I detta fall ser vi att det gick snabbt att träna modellerna, inte ens en sekund tog det. För stora dataset och komplexa modeller kan modellträningen ta längre tid där det kan dröja några minuter, timmar eller dagar. Vi fortsätter att inspektera ytterligare kolumner från den *grid search* vi genomfört.

```
df_gs_results.iloc[:, [5, 6, 7, 8]]
```

	params	split0_test_score	split1_test_score	mean_test_score
0	{'fit_intercept': True}	-59.095576	-28.876354	-43.985965
1	{'fit_intercept': False}	-47.703118	-28.890138	-38.296628

Från tabellen ovan ser vi att vi har två resultat, `split0_test_score` och `split1_test_score`. Detta eftersom vi satte hyperparametern `cv=2` i koden tidigare. Högst resultat, det vill säga det värde som är närmast noll (eftersom vi har negativa värden), har den andra modellen där `fit_intercept = False`. Därför bör vi välja den modellen. Från resultattabellen nedan ser vi kolumnen `rank_test_score` och att den modellen alltså är rankad som nummer ett.

```
df_gs_results.iloc[:, [5, 8, 10]]
```

	params	mean_test_score	rank_test_score
0	{'fit_intercept': True}	-43.985965	2
1	{'fit_intercept': False}	-38.296628	1

Läsaren uppmanas att själv inspektera resultaten i sin helhet genom att exekvera nedanstående kod. Notera, vi visar inte fullständiga resultat från koden på grund av att de inte ryms på bokens sida.

```
pd.DataFrame(grid_search.cv_results_)
```

Läser vi den officiella dokumentationen för `GridSearch` från *scikit-learn* ser vi att följande hyperparametrar finns:

```
class sklearn.model_selection.GridSearchCV(estimator, param_grid,
    ↪ *, scoring=None, n_jobs=None, refit=True, cv=None, verbose=0,
    ↪ pre_dispatch='2*n_jobs', error_score=nan,
    ↪ return_train_score=False)
```

Vi ser där att hyperparametern `refit` har default-värdet `True`. Fortsätter vi läsa dokumentationen ser vi att det innebär följande: “*Refit an estimator using the best found parameters on the whole dataset.*” När vi genomfört vår *grid search*, det vill säga använt `.fit()` metoden, så tränas modellen om med de bästa hyperparametrarna på hela datasetet och sparas så vi kan använda den, exempelvis för att genomföra prediktioner.

```
from sklearn.model_selection import GridSearchCV

model = LinearRegression()

hyperparameters = {'fit_intercept': [True, False]}

grid_search = GridSearchCV(estimator=model,
    ↪ param_grid=hyperparameters, scoring='neg_mean_squared_error',
    ↪ cv=3)

grid_search.fit(example_data[["x"]], example_data["y"]) ①

grid_search.predict([[6]]) ②
```

- ① En *grid search* genomförs och när de optimala hyperparametrarna hittats så tränas modellen om på hela datasetet med de optimala hyperparametrarna och sparas. I detta fallet sparas den optimala modellen i variabeln `grid_search`.
- ② Vi använder vår optimala modell för att genomföra en prediktion. Notera, vi genomför alltså en prediktion för $x = 6$ och får ett värde enligt nedan, att resultatet är rimligt kan vi verifiera med hjälp av Figur 1.7.

```
array([4.78915663])
```

Koden nedan demonstrerar ytterligare några inspektioner som kan utföras för en `GridSearch`.

```

print("Best Hyperparameters from GridSearchCV:",
      ↪ grid_search.best_params_)
print("Best Cross-Validation MSE:", -grid_search.best_score_)

best_model = grid_search.best_estimator_
print(f"Best Model Coefficients. Intercept:
      ↪ {round(best_model.intercept_, 2)}, Slope:
      ↪ {round(best_model.coef_[0], 2)}")

```

```

Best Hyperparameters from GridSearchCV: {'fit_intercept': False}
Best Cross-Validation MSE: 28.554233641184407
Best Model Coefficients. Intercept: 0.0, Slope: 0.8

```

Vi avslutar detta avsnitt med att nämna att det finns en klass `RandomizedSearchCV` i *scikit-learn*. Denna klass är motsvarigheten till `GridSearchCV` men skillnaden är att den slumpmässigt väljer vilka hyperparametervärden som utvärderas. I praktiken kan den vara användbar om det finns ett stort antal möjliga hyperparametervärden att undersöka. Den intresserade läsaren kan läsa dokumentationen från *scikit-learn* för `RandomizedSearchCV`.

1.3.6 Kategorisk data

När vi arbetar med ML-modeller, exempelvis den linjära regressionsmodellen, så behöver modellerna numerisk indata. Trots detta så vet vi från verkligheten att kategorisk data som antar värden i olika kategorier såsom färg och kön kan vara viktiga. Hur kan vi inkorporera sådan data? Vi omvandlar den kategoriska datan till numerisk data och kan därefter använda den i våra modeller. När vi arbetar med kategorisk data kan vi skilja mellan nominal data och ordinal data.

Nominal data kan anta ett fixt antal specifika värden där värdena saknar inbördes rangordning. Exempelvis {röd, grön, blå, svart} kan vi tolka som nominal data. Denna typ av data kan vi omvandla till numerisk data genom metoder som kallas för *one-hot-encoding* och *dummy-variable-encoding*. Ordinal data kan anta ett fixt antal specifika värden där värdena har en inbördes rangordning. Om vi exempelvis har data från en kundundersökning och frågar kunderna om deras upplevelse så kan möjliga svar vara {Ej nöjd, Neutral, Nöjd}. Denna typ av data kan vi omvandla till numerisk data genom en metod som kallas för *ordinal-encoding*. Notera, huruvida datan är nominal eller ordinal beror på situationen och hur vi väljer att tolka den. Tidigare sade vi att färgerna {röd, grön, blå, svart} kan tolkas som att de är på nominal skala. Om vi däremot exempelvis

vet att röda bilar är mer populära än gröna bilar som i sin tur är mer populära än blåa bilar som i sin tur är mer populära än svarta bilar, då finns det plötsligt en inbördes rangordning och datan kan då tolkas som att den är på ordinal skala. Här behöver alltså den som genomför ML-modellering själv göra ett val vilket alltså är en del av konsten.

I koden nedan skapar vi exempeldata för demonstrationssyfte och sparar det i variabeln `nominal_df`.

```
nominal_data = {'färg': ['röd', 'grön', 'blå', 'svart', 'röd',
    ↪ 'grön', 'grön', 'röd']}
nominal_df = pd.DataFrame(nominal_data)
nominal_df
```

	färg
0	röd
1	grön
2	blå
3	svart
4	röd
5	grön
6	grön
7	röd

Datan från tabellen ovan kommer vi nu att omvandla med hjälp av *one-hot-encoding*. Eftersom vi har fyra distinkta kategorier {röd, grön, blå, svart} kommer vi att få fyra nya kolumner efter att vi har genomfört en *one-hot-encoding*. Radsumman för varje rad blir ett eftersom en färg endast kan vara en färg i taget. För att genomföra vår *one-hot-encoding* använder vi *Pandas* funktionen `get_dummies()`. Funktionen heter så eftersom variabler/kolumner som endast kan anta värdet 0 eller 1 kallas för dummyvariabel.

```
oh_encoding = pd.get_dummies(nominal_df, dtype = int)
oh_encoding
```

	färg_blå	färg_grön	färg_röd	färg_svart
0	0	0	1	0

	färg_blå	färg_grön	färg_röd	färg_svart
1	0	1	0	0
2	1	0	0	0
3	0	0	0	1
4	0	0	1	0
5	0	1	0	0
6	0	1	0	0
7	0	0	1	0

Ovan demonstrerade vi *one-hot-encoding*. Nu kommer vi demonstrera *dummy-variable-encoding*. Generellt gäller att om vi använder *dummy-variable-encoding* för en variabel med k kategorier så kommer vi få k-1 nya variabler/kolumner. I vårt fall har vi fyra kategorier och får därför 3 nya variabler. Detta räcker eftersom i kodexemplet nedan markeras grön med kolumnen `färg_grön = 1`, röd med kolumnen `färg_röd = 1`, svart med kolumnen `färg_svart = 1` och i de fall alla tre kolumnerna har värdet 0 vet vi att det är färgen blå som åsyftas. Se exempelvis raden med index 2 där alla kolumnerna har värdet 0, då vet vi att det är färgen blå.

```
dummy_encoding = pd.get_dummies(nominal_df, dtype = int,
    ↪ drop_first = True)
dummy_encoding
```

①

- ① Notera att för såväl *one-hot-encoding* som för *dummy-variable-encoding* kan vi använda pandas funktionaliteten `pd.get_dummies()`. I det fallet vi önskar genomföra en *dummy-variable-encoding* specificerar vi hyperparametern `drop_first = True`, precis som vi gjorde i detta kodexemplet.

	färg_grön	färg_röd	färg_svart
0	0	1	0
1	1	0	0
2	0	0	0
3	0	0	1
4	0	1	0
5	1	0	0
6	1	0	0
7	0	1	0

Men vad är skillnaden mellan *one-hot-encoding* och *dummy-variable-encoding*? Rent informationsmässigt ger metoderna samma information men när vi ska använda nominal data i den linjära regressions-modellen så ska vi använda *dummy-variable-encoding* medan vi för övriga modeller använder *one-hot-encoding*. Varför det är så går att matematiskt motivera men det är inget vi går djupare in på i denna bok.

Nedan demonstrerar vi *ordinal encoding* för kolumnen `Pris` där vi ser att det finns en tydlig rangordning, `låg`, `medel` och `hög`.

```
ordinal_data = {'Produkt': ['XXX', 'YYY', 'XXY', 'YXX', 'YXX'],
                'Pris': ['hög', 'medel', 'låg', 'hög', 'låg']}
ordinal_df = pd.DataFrame(ordinal_data) ①
ordinal_df
```

- ① Vi skapar exempeldata för demonstrationssyfte och sparar den i variabeln `ordinal_df`.

	Produkt	Pris
0	XXX	hög
1	YYY	medel
2	XXY	låg
3	YXX	hög
4	YXX	låg

```
mapping = {"låg": 1, "medel": 2, "hög": 3} ①
ordinal_df["Pris"] = ordinal_df["Pris"].map(mapping) ②
ordinal_df
```

- ① Vi specificerar vilken korrespondens vi önskar. Exempelvis kommer `låg` korrespondera med 1.
- ② Vi använder `map()` funktionen från Pandas för att ersätta värdena `låg`, `medel` och `hög` med deras korresponderade värden sparade i variabeln `mapping` i detta fall.

	Produkt	Pris
0	XXX	3
1	YYY	2

	Produkt	Pris
2	XXY	1
3	XYX	3
4	YXX	1

Önskar vi lägga till en kolumn istället för att ersätta den ursprungliga **Pris** kolumnen kan vi göra det också.

```
ordinal_df_2 = pd.DataFrame(ordinal_data)

ordinal_df_2['PrisOrdinalKodad'] =
    ↪ ordinal_df_2['Pris'].map(mapping)
ordinal_df_2
```

	Produkt	Pris	PrisOrdinalKodad
0	XXX	hög	3
1	YYY	medel	2
2	XXY	låg	1
3	XYX	hög	3
4	YXX	låg	1

I exemplen ovan har vi arbetat med data som är i formatet *Pandas dataframe*. Arbetar vi med data som är i *NumPy array* format kan vi använda funktionerna **OneHotEncoder** och **OrdinalEncoder** från *scikit-learn*. Den intresserade läsaren uppmanas att läsa dokumentationen. Dessa funktionerna går att använda för *Pandas dataframes* också men kommer då konvertera den ursprungliga *dataframen* till en *NumPy array*. Vi demonstrerar detta nedan.

```
nominal_df
```

	färg
0	röd
1	grön

	färg
2	blå
3	svart
4	röd
5	grön
6	grön
7	röd

```
from sklearn.preprocessing import OneHotEncoder
OneHotEncoder(sparse_output=False).fit_transform(nominal_
↪ df[['färg']])
```

```
array([[0., 0., 1., 0.],
       [0., 1., 0., 0.],
       [1., 0., 0., 0.],
       [0., 0., 0., 1.],
       [0., 0., 1., 0.],
       [0., 1., 0., 0.],
       [0., 1., 0., 0.],
       [0., 1., 0., 0.],
       [0., 0., 1., 0.]])
```

1.3.7 *Feature engineering*

Feature engineering handlar om vilka variabler som ska väljas (variabelselektion) i en modell, skapandet av nya variabler och transformering av variabler. Generellt sett åsyftas de oberoende variablerna (*features*) men även exempelvis transformering av den beroende variabeln kan göras. Nedan skapar vi exempeldata som vi kommer använda oss av.

```
example_data = {'pris (tkr)': [4200, 2500, 7900, 6100, 6500],
                 'kvm': [120, 80, 250, 200, 150],
                 'antal rum': [4, 3, 7, 5, 4],
                 'byggår': [2005, 2016, 1980, 1990, 2020]}
example_data = pd.DataFrame(example_data)
example_data
```

	pris (tkr)	kvm	antal rum	byggår
0	4200	120	4	2005
1	2500	80	3	2016
2	7900	250	7	1980
3	6100	200	5	1990
4	6500	150	4	2020

När vi skapar en modell behöver vi ta ställning till vilka variabler som ska användas, detta benämns *variabelselektion*. I en modell kanske vi väljer att modellera hur variabeln **pris (tkr)** beror på variabeln **kvm** som står för kvadratmeter. I en annan modell kanske vi väljer att modellera hur variabeln **pris (tkr)** beror på variablerna **kvm** och **antal rum**. När vi tränat dessa två modeller på träningsdatan så utvärderar vi dem på valideringsdatan och går vidare med den som presterar bäst.

Variabelselektion är generellt sett mycket viktigt eftersom oavsett hur komplicerade modeller vi använder oss av så kommer de inte att prestera bra om fel variabler används. “*Shit in, shit out*” är ett uttryck för detta, det vill säga stoppar vi in dåliga variabler i modellerna får vi ut dåliga prediktioner. Hur väljer vi då vilka variabler som ska användas? Den kanske viktigaste metodiken är att använda sitt omdöme och eventuell teori som säger att en viss variabel påverkar den beroende variabeln (y). Exempelvis finns det forskning som visar att motion påverkar hälsan, om vi då hade velat modellera en människas hälsa så finns det teoretiska skäl till att använda motion som en oberoende variabel. Om vi tror att flera olika uppsättningar av variabler kan vara användbara så kan vi träna olika modeller på träningsdatan och därefter på valideringsdatan utvärdera vilken som presterar bäst. Vi betonar här att vi behöver använda vårt omdöme på vilka uppsättningar av variabler som används. Detta eftersom ifall vi exempelvis tränar 1000 olika modeller på träningsdatan så är sannolikheten troligtvis ganska hög att det av ren slump blir någon modell som presterar bra. Det finns även algoritmer som kan användas för att genomföra variabelselektion på ett mer automatiserat sätt. Exempelvis, när vi lär oss om beslutsträd och *random forest* modeller senare i boken, kommer vi se att dessa modeller kan användas för att bedöma hur viktiga olika variabler är genom något som benämns för *feature importance* (gåss igenom i Avsnitt 4.3.5). Poängen att man kan använda de variabler som har högst *feature importance* eftersom de enligt modellen är viktigast. Den läsare som vill fördjupa sig inom algoritmisk variabelselektion kan googla på *Feature selection* och läsa *scikit-learns* dokumentation, se följande länk: https://scikit-learn.org/stable/modules/feature_selection.html. Vi poängterar dock att metoden där omdömet och teori används är viktig, förkasta alltså inte omdömet bara för att eventuella algoritmer används för variabelselektion.

i Man kan fråga sig om det inte vore bäst att helt enkelt använda alla tillgängliga variabler i en modell? Rent generellt är det en dålig idé eftersom modellen troligtvis kommer generalisera dåligt till ny, osedd data. Den blir alltså överanpassad.

En generell princip inom modellering benämns *principle of parsimony* och den säger att givet en viss prestationsnivå så försöker man ha en så enkel modell som möjligt. Rent generellt, inom vetenskapliga sammanhang, benämns det Ockhams rakkniv.

Vi kan även skapa nya variabler. Se kodexemplet nedan där vi skapar en ny variabel som beräknar genomsnittligt antal kvadratmeter per rum och en dummyvariabel som är 1 ifall huset är byggt senare än år 2000 och 0 annars. Vi kanske vet att hus byggda efter år 2000 är mer attraktiva då ny typ av byggteknik började användas. Vi ser alltså att vi kan använda vår domänkunskap, om hus i detta fall, för att skapa en ny variabel. Det gäller generellt att domänkunskap är användbart i modellering. Ibland kanske vi redan har det från början av projektet och om vi inte har det lär vi behöva läsa in oss för att faktiskt få det, åtminstone grundläggande sådan.

```
example_data['snitt_kvm_per_rum'] = example_data['kvm'] /  
    ↪ example_data['antal rum']  
  
example_data['ny_husmodell'] = (example_data['byggår'] >  
    ↪ 2000).astype(int)  
  
example_data
```

	pris (tkr)	kvm	antal rum	byggår	snitt_kvm_per_rum	ny_husmodell
0	4200	120	4	2005	30.000000	1
1	2500	80	3	2016	26.666667	1
2	7900	250	7	1980	35.714286	0
3	6100	200	5	1990	40.000000	0
4	6500	150	4	2020	37.500000	1

Variablerna vi skapade ovan kan därefter användas i en modell och genom att utvärdera modellen på valideringsdatan kan vi se om variablerna är till hjälp.

Ett annat sätt att få nya variabler är att samla in mer data. Hade vi exempelvis velat prediktera huspriser så vet vi att geografiskt läge och närhet till havet är två faktorer som kan ha stor påverkan på priset. I praktiken kan det vara en kostsam process att samla in mer data och en bedömning kring om det är värt det eller inte behöver göras.

Variabler kan även transformeras. Inom ML standardiserar man ofta variabler för att de ska vara på samma skala. Logiken är att om variablerna är på samma skala kan det bli lättare för modellen att se mönster. I koden nedan demonstreras **StandardScaler** i *scikit-learn* som standardiserar variablerna till att ha medelvärdet 0 och standardavvikelsen 1. Notera, när vi säger att variablerna har medelvärdet 0 och standardavvikelsen 1 menar vi att respektive kolumn har medelvärdet 0 och standardavvikelse 1.

```
from sklearn.preprocessing import StandardScaler
import numpy as np

example_data = np.array([[1.0, 2.0], [3.0, 6.0], [5.0, 10.0]]) ①

scaler = StandardScaler() ②
example_data_scaled = scaler.fit_transform(example_data) ③

print("Original data:\n", example_data)
print("Scaled data:\n", example_data_scaled)
print("Medelvärde efter skalning:", np.mean(example_data_scaled,
↪ axis=0)) ④
print("Standardavvikelse efter skalning:",
↪ np.std(example_data_scaled, axis=0)) ⑤
```

- ① Vi skapar exempeldata för demonstrationssyfte.
- ② Vi instantierar **StandardScaler** från *scikit-learn*.
- ③ Vi transformerar vår exempeldata med **StandardScaler** till att ha medelvärdet 0 och standardavvikelsen 1. Resultatet sparas i den nya variabeln **example_data_scaled**.
- ④ Vi demonstrerar att respektive kolumn nu har medelvärdet 0.
- ⑤ Vi demonstrerar att respektive kolumn nu har standardavvikelsen 1.

Original data:

```
[[ 1.  2.]
 [ 3.  6.]
 [ 5. 10.]]
```

Scaled data:

```

[[-1.22474487 -1.22474487]
 [ 0.         0.         ]
 [ 1.22474487  1.22474487]]
Medelvärde efter skalning: [0. 0.]
Standardavvikelse efter skalning: [1. 1.]

```

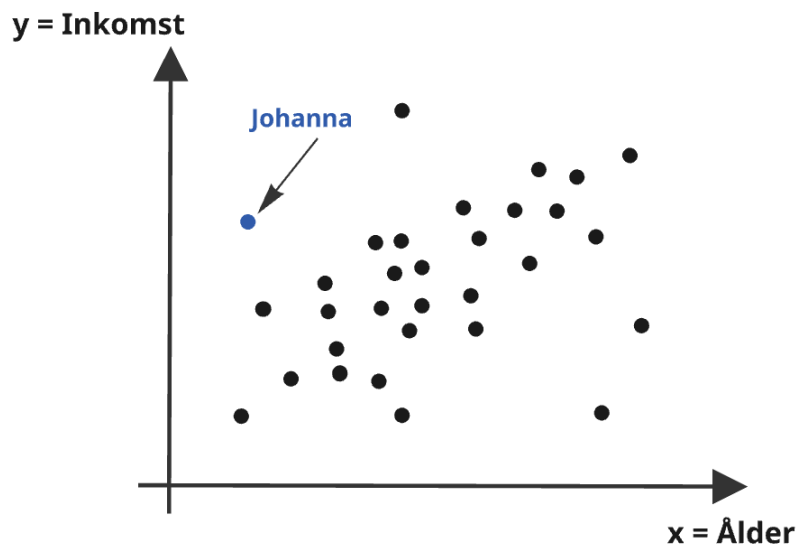
Det går att standardisera variabler på fler sätt, exempelvis kan den intresserade läsaren läsa *scikit-learn* dokumentationen för `MinMaxScaler` som gör att variablerna befinner sig i ett visst intervall, exempelvis mellan 0 – 1. Inom ML förekommer det också att variabler logaritmeras (log-transformeras), exempelvis för att de ska se mer normalfördelade ut. Vi går dock inte djupare in på det utan nämner endast det för den läsare som har kunskaper i mer avancerad statistik. Avslutningsvis nämner vi att senare i boken, Kapitel 5, kommer vi lära oss en modell som heter *Principal Component Analysis (PCA)* som också möjliggör oss att transformera variabler.

1.4 Tvärsnittsdata, tidsseriedata och paneldata

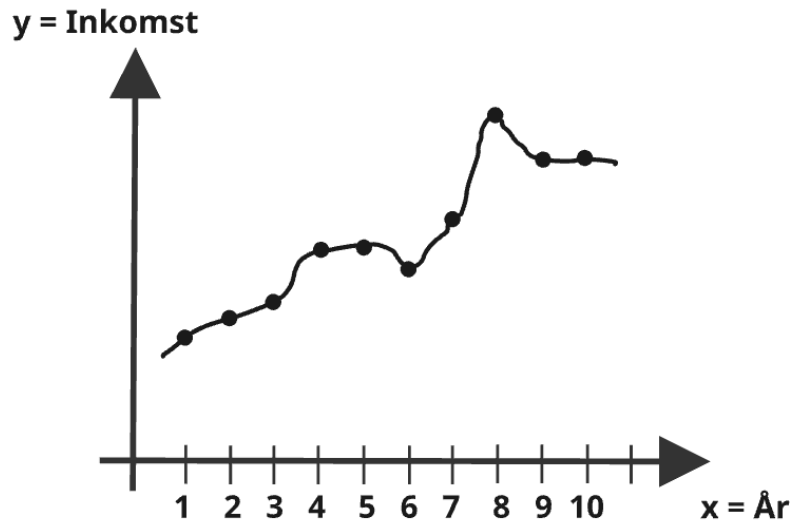
Data kan kategoriseras på olika sätt. Bra kategorier som är bra att känna till är *tvärsnittsdata*, *tidsseriedata* och *paneldata*.

- Tvärsnittsdata är sådan data där observationer för olika individer (det kan exempelvis vara människor, företag eller länder) samlas in vid *en tidpunkt*. Se Figur 1.8 på hur tvärsnittsdata kan se ut.
- Tidsseriedata är sådan data där observationer över tid samlas in för en individ (det kan exempelvis vara människor, företag eller länder). Se Figur 1.9 på hur tidsseriedata kan se ut.
- Paneldata är sådan data där observationer för individer (det kan exempelvis vara människor, företag eller länder) samlas in över tid. Notera alltså att tvärsnittsdata och tidsseriedata kan betraktas som specialfall av paneldata. Tvärsnittsdata är en tidpunkt från paneldatan och tidsseriedatan är en individ från paneldatan. Se Figur 1.10 på hur paneldata kan se ut.

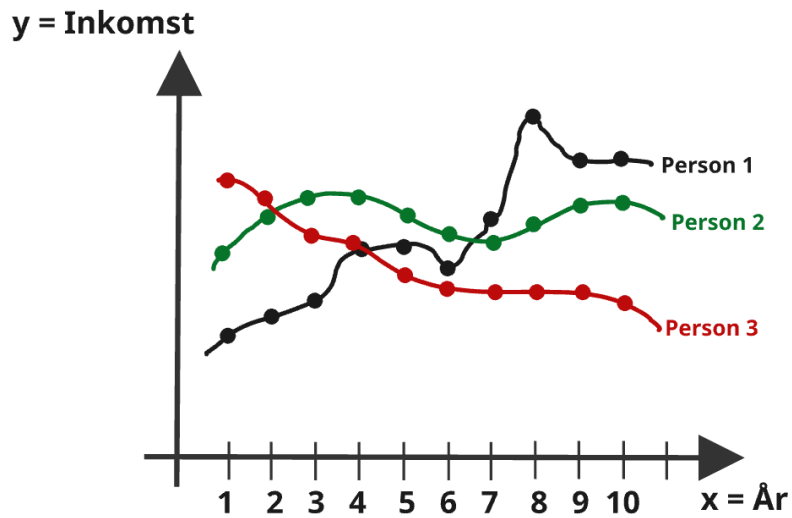
I boken kommer vi mestadels arbeta med tvärsnittsdata, bland annat när vi tillämpar regressionsmodeller och klassificeringsmodeller. Tidsseriedata kommer vi mycket översiktligt nämna i Kapitel 9. Paneldata kommer vi inte gå in på i denna bok. Det finns mycket information på bland annat internet där analysmetoder och modeller för respektive data-kategori behandlas som den intresserade läsaren kan läsa in sig på.



Figur 1.8: Exempel på hur tvärsnittsdata kan se ut. Vi har ålder och inkomst för olika personer. Exempelvis har vi i figuren markerat en observation där personen heter Johanna.



Figur 1.9: Ett exempel på hur tidsseriedata kan se ut. Vi följer inkomsten över tid för en person.



Figur 1.10: Ett exempel på hur paneldata kan se ut. Vi har följt inkomsterna över tid för tre olika personer.

i Tidsserier - ARMA

Den läsare som vill lära sig om tidsseriemodeller kan börja kolla på de modeller som benämns för *ARMA* (*Autoregressive Moving Average*) där *AR*- och *MA*-modellerna är specialfall.

1.5 Övningsuppgifter

Övningsuppgifter för kapitlet finns på bokens hemsida:
https://github.com/AntonioPrgomet/ai_tillaempad_ml

Kapitel 2

Ett ML-projekt från början till slut

I detta kapitel kommer vi börja med att gå igenom en praktisk checklista som kan vara till hjälp när vi genomför ett ML-projekt. Denna checklista kommer vi sedan att använda när vi genomför ett kodexempel, från början till slut, där huspriser i Kalifornien kommer modelleras. Därefter tar vi ta upp några vanligt förekommande utmaningar inom ML-projekt. Kapitlet avslutas med ett avsnitt om *scikit-learn*, ett mycket populärt och använt bibliotek inom AI/ML branschen som vi kommer att använda genom hela boken.

2.1 En checklista för Maskininlärning

Vi presenterar här en checklista som ska ge en orientering över möjliga steg att genomföra i ett ML-projekt. Checklistan har sju steg och det är lätt att få intrycket att dessa steg görs i en rak progression - vi gör steg ett, går vidare till steg två för att därefter fortsätta med steg tre och så vidare. Så är det inte. Ofta kan vi behöva hoppa mellan stegen, exempelvis för att vi fått en insikt som påverkar vår analys från föregående steg eller för att det vi egentligen ville göra inte är möjligt. Så i verkligheten är vi beredda på att vi behöver arbeta iterativt där vi går “fram-och-tillbaka” och hoppar mellan stegen. Notera också att checklistan ska utgöra ett stöd i vårt arbete och inget som vi varken behöver eller ska följa slaviskt.